

Homework #6 Solution
Spring of 2008

* This solution contains several problems in HW6.

Topic: Graph algorithm

1. Do at least 50 point from:

25.1-6

Essentially we use the fact that $L \bullet W = L$ where L is the completed matrix of shortest path weights.

So our argument is that if shortest path from i to j is finite, there must be some $k (\neq j)$ such that $\delta(i, j) = \delta(i, k) + w_{kj}$. Here k is the predecessor of the path from i to j . Here is the code.

CALCULATE-PRED-MATRIX(L, W)

```
1   $n \leftarrow \text{rows}[L]$ 
2  Let  $\Pi = (\pi_{ij})$  be a  $n \times n$  matrix
3  for  $i \leftarrow 1$  to  $n$ 
4      do for  $j \leftarrow 1$  to  $n$ 
5          do if  $i = j$  or  $l_{ij} = \infty \triangleright$  no predecessor
6              then  $\pi_{ij} \leftarrow \text{NIL}$ 
7              else  $\pi_{ij} \leftarrow \text{NIL}$ 
8                  for  $k \leftarrow 1$  to  $n$ 
9                      do if  $l_{ij} = l_{ik} + w_{kj}$  and  $k \neq j$ 
10                          then  $\pi_{ij} \leftarrow k$ 
11  return  $\Pi$ 
```

Running time of the algorithm will be $O(n^3)$, because there are three nested loops that loops n times.

25.2-5 *From UTDallas*

YES. The predecessor matrix Π may be different which means the shortest paths may be different but the lengths of those paths are the same.

25.2 – 8

We wish to compute the transitive closure of a directed graph $G = (V, E)$. Construct a new graph $G^* = (V, E^*)$ where E^* is initially empty. For each vertex v traverse the graph G adding edges for every node encountered in E^* . This takes $O(VE)$ time.

25.2-9 *From Stanford University*

Answer: If we have an algorithm to compute the transitive closure of a directed acyclic graph, we can use it to calculate the transitive closure of any directed graph. We first decompose the directed graph G into its strongly connected component graph G_{SCC} . This runs in time $O(V + E)$ and results in a graph with at most V vertices and E edges. We keep track of which set of vertices in G map to each vertex in G_{SCC} . Call this $g(v)$ which returns a strongly connected component in G_{SCC} . We then call the transitive closure algorithm on the graph G_{SCC} . This runs in time $O(f(|V|, |E|))$. All that is left is to convert back from the strongly connected component graph's transitive closure to G 's transitive closure. There is an edge (u, v) in the transitive closure of G if and only if there is an edge $(g(u), g(v))$ in the graph G_{SCC} . The forward direction is easy to see. If there is a path from u to v in G , then there must be a path between the component that u belongs to and the one that v belongs to in G_{SCC} . The reverse direction is equally easy. If there is a path between u 's strongly connected component and v 's strongly connected component in G_{SCC} , then there is a path between some vertex a in $g(u)$ and some vertex b in $g(v)$ in G . But, since u and a are in the same SCC, there is a path from $u \rightsquigarrow a$ and since v and b are in the same SCC, there is a path from $v \rightsquigarrow b$. Therefore, there is a path $u \rightsquigarrow a \rightsquigarrow b \rightsquigarrow v$ in G .

To actually calculate the transitive closure of G this way would require time V^2 since we'd need to look at all pairs. Instead, we will look at the transitive closure of G_{SCC} . For each edge (a, b) in the transitive closure of G_{SCC} , we assume that we know the set of vertices in G that map to a and b , call these sets $g^{-1}(a)$ and $g^{-1}(b)$ respectively. We will add the edge (x, y) for all vertices $x \in g^{-1}(a)$ and $y \in g^{-1}(b)$. This will take time equal to the number of edges in the transitive closure of G , $O(E^*)$.

Therefore the total running time for the algorithm is the time to build the strongly connected components ($O(V + E)$), the time to run the transitive closure algorithm on G_{SCC} ($O(f(|V|, |E|))$) and the time to convert back to the transitive closure of G ($O(E^*)$). Since we know that $E < E^*$, the total time is $O(f(|V|, |E|) + V + E^*)$.

25.3-5 *From the internet* <http://algorithmics.bu.edu>

By definition of $\hat{w}$, given a path $p = \{v_0, \dots, v_k\}$, $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$.

Assume next that both v_0 and v_k are in $c = \{v_0, \dots, v_k, \dots, v_{m-1}\}$ (the zero weight cycle), and define $p_1 = \{v_0, \dots, v_k\}$, $p_2 = \{v_k, \dots, v_{m-1}, v_0\}$ then $\hat{w}(p_1) = w(p_1) + h(v_0) - h(v_k)$, and $\hat{w}(p_2) = w(p_2) + h(v_k) - h(v_0)$. Combining these two equations and since $w(p_1) + w(p_2) = 0$ we get $\hat{w}(p_1) = -\hat{w}(p_2)$. Since the values of $\hat{w}$ are non-negative for all edges (should be after reweighing), it follows that for all edges of the 0-cycle $\hat{w}(e) = 0$.

25-2 *From the web*

- a) INSERT and DECREASE-KEY have running time equal to the height of the heap, which is

$$\lg_d n = \lg n / \lg d = \lg n / \alpha \lg n = 1/\alpha.$$

EXTRACT-MIN has running time $d \lg_d n = n^\alpha / \alpha$, because the procedure needs to scan the d children at every node.

- b) Let $d = n^\epsilon$. The running time of Dijkstra's algorithm is

$$O(V \cdot V^\alpha / \alpha + E / \alpha) = O(V^{1+\epsilon} + E) = O(E).$$

- c) Run the algorithm in (b) V times. Each time select a different vertex in G as the source.
- d) Our modification to Dijkstra's algorithm only affects how the min-priority heap is implemented. We can still use Johnson's algorithm to deal with negative-weight edges.

2. Do at least 50 points from:

26.1-6

The flow sum $f_1 + f_2$ satisfies skew symmetry and flow conservation, but might violate the capacity constraint.

We give proofs for skew symmetry and flow conservation and an example that shows a violation of the capacity constraint. Let $f(u, v) = (f_1 + f_2)(u, v)$.

For skew symmetry:

$$\begin{aligned} f(u, v) &= f_1(u, v) + f_2(u, v) \\ &= -f_1(v, u) - f_2(v, u) \quad (\text{skew symmetry}) \\ &= -(f_1(v, u) + f_2(v, u)) \\ &= -f(v, u). \end{aligned}$$

For flow conservation, let $u \in V - \{s, t\}$:

$$\begin{aligned} \sum_{v \in V} f(u, v) &= \sum_{v \in V} (f_1(u, v) + f_2(u, v)) = \sum_{v \in V} f_1(u, v) + \sum_{v \in V} f_2(u, v) \\ &= 0 + 0 \quad (\text{flow conservation}) \\ &= 0. \end{aligned}$$

For the capacity constraint, let $V = \{s, t\}$, $E = \{(s, t)\}$, and $c(s, t) = 1$. Let $f_1(s, t) = f_2(s, t) = 1$. Then f_1 and f_2 obey the capacity constraint, but $(f_1 + f_2)(u, v) = 2$, which violates the capacity constraint.

26.1-7

To see that the flows form a convex set, we show that if f_1 and f_2 are flows, then so is $\alpha f_1 + (1 - \alpha)f_2$ for all α such that $0 \leq \alpha \leq 1$.

For the capacity constraint, first observe that $\alpha \leq 1$ implies that $1 - \alpha \geq 0$. Thus, for any $u, v \in V$, we have

$$\begin{aligned}\alpha f_1(u, v) + (1 - \alpha)f_2(u, v) &\geq 0 \cdot f_1(u, v) + 0 \cdot f_2(u, v) \\ &= 0.\end{aligned}$$

Since $f_1(u, v) \leq c(u, v)$ and $f_2(u, v) \leq c(u, v)$, we also have

$$\begin{aligned}\alpha f_1(u, v) + (1 - \alpha)f_2(u, v) &\leq \alpha \cdot c(u, v) + (1 - \alpha) \cdot c(u, v) \\ &= (\alpha + (1 - \alpha))c(u, v) \\ &= c(u, v).\end{aligned}$$

For skew symmetry, we have $f_1(u, v) = -f_1(v, u)$ and $f_2(u, v) = -f_2(v, u)$ for any $u, v \in V$. Thus, we have

$$\begin{aligned}\alpha f_1(u, v) + (1 - \alpha)f_2(u, v) &= -\alpha f_1(v, u) - (1 - \alpha)f_2(v, u) \\ &= -(\alpha f_1(v, u) + (1 - \alpha)f_2(v, u)).\end{aligned}$$

For flow conservation, observe that since f_1 and f_2 obey flow conservation, we

Have $\sum_{v \in V} f_1(u, v) = 0$ and $\sum_{v \in V} f_2(u, v) = 0$ for any $u \in V - \{s, t\}$. Thus,

$$\begin{aligned}\sum_{v \in V} (\alpha f_1(u, v) + (1 - \alpha)f_2(u, v)) &= \alpha \sum_{v \in V} f_1(u, v) + (1 - \alpha) \sum_{v \in V} f_2(u, v) \\ &= \alpha \cdot 0 + (1 - \alpha) \cdot 0 \\ &= 0.\end{aligned}$$

26.2-9

For any two vertices u and v in G , you can define a flow network G_{uv} consisting of the directed version of G with all edge capacities set to 1, $s = u$, and $t = v$.

(G_{uv} has $O(V)$ vertices—actually, $|V|$ —and $O(E)$ edges, as required. We want all capacities to be 1 so that the number of edges crossing a cut equals the capacity of the cut.) Let f_{uv} denote a maximum flow in G_{uv} .

We claim that for any $u \in V$, the edge connectivity k equals $\min_{v \in V - \{u\}} |f_{uv}|$. We'll show below that this claim holds. Assuming that it holds, we can find k as follows:

EDGE-CONNECTIVITY(G)

select any vertex $u \in V$

for each vertex $v \in V - \{u\}$ $\triangleright |V| - 1$ iterations

do set up the flow network G_{uv} as described above

 find the maximum flow f_{uv} on G_{uv}

return the minimum of the $|V| - 1$ max-flow values: $\min_{v \in V - \{u\}} |f_{uv}|$

The claim follows from the max-flow min-cut theorem and how we chose capacities so that the capacity of a cut is the number of edges crossing it. We prove

that $k = \min_{v \in V - \{u\}} |f_{uv}|$, for any $u \in V$ by showing separately that k is at least this

minimum and that k is at most this minimum.

• Proof that $k \geq \min_{v \in V - \{u\}} |f_{uv}|$:

Let $m = \min_{v \in V - \{u\}} |f_{uv}|$. Suppose we remove only $m - 1$ edges from G . For

any vertex v , by the max-flow min-cut theorem, u and v are still connected.

(The max flow from u to v is at least m , hence any cut separating u from v has capacity at least m , which means at least m edges cross any such cut. Thus at least one edge is left crossing the cut when we remove $m-1$ edges.) Thus every node is connected to u , which implies that the graph is still connected. So at least m edges must be removed to disconnect the graph—i.e., $k \geq \min_{v \in V - \{u\}} |f_{uv}|$.

• Proof that $k \leq \min_{v \in V - \{u\}} |f_{uv}|$:

Consider a vertex v with the minimum $|f_{uv}|$. By the max-flow min-cut theorem, there is a cut of capacity $|f_{uv}|$ separating u and v . Since all edge capacities are 1, exactly $|f_{uv}|$ edges cross this cut. If these edges are removed, there is no path from u to v , and so our graph becomes disconnected. Hence $k \leq \min_{v \in V - \{u\}} |f_{uv}|$.

• Thus, the claim that $k = \min_{v \in V - \{u\}} |f_{uv}|$, for any $u \in V$ is true.